Demo - OpenCode

Yogesh Haribhau Kulkarni

Workshop Overview

What we will build: A chatbot with two interfaces — CLI and Streamlit — powered by LangChain + Groq LLM.

What we will learn:

- How to use OpenCode across the **full SDLC** — from specs to deployment
- **Custom commands**, **skills**, **rules**, **MCP servers**, **agents**
- Writing code, tests, and docs — *all from the terminal*
- How to be precise with prompts so OpenCode does what you intend

Duration: 90–120 minutes end-to-end.

SDLC Stages We Will Cover

Stage	Activity	OpenCode Feature
1	Project bootstrap & init	`/init`, AGENTS.md
2	Requirements & specs	Custom agent, Markdown
3	Architecture design	Plan mode, `@workspace`
4	Implementation	Build mode, `@file:`
5	Testing (unit + synthetic)	Custom command, MCP
6	Debugging	Debug agent, `/undo`
7	Refactoring	Rules, skills
8	Documentation	Docs agent
9	Code review	Review agent
10	GitHub workflow	MCP GitHub server

Each stage = one demo block. Run them sequentially or jump to any stage.

Stage 0: Environment Setup

Pre-requisites Checklist

Before starting, verify:

- Start with empty folder, switch to that
- Conda env installed, activated: `conda --version`
- OpenCode installed: `opencode --version`
- Node.js $\geq$ 18 for MCP servers: `node --version`
- Python $\geq$ 3.10 and pip: `python --version`
- (Optional) Groq API key: `console.groq.com` (free tier available)
- A GitHub Personal Access Token (for MCP GitHub stage)
- Git configured: `git config --global user.name "Your Name"`

Set environment variables (add to `.bashrc`/`.zshrc`):

```
export GROQ_API_KEY="gsk_..." # or your preferred
    LLM
export GITHUB_TOKEN="ghp_..."
```

Install Python deps we'll use:

```
pip install langchain langchain-groq streamlit
    rich pytest
pip install pytest-mock faker
```

Connect OpenCode to Your LLM Provider

Step 1 — Launch OpenCode and connect Groq (or Anthropic/OpenAI):

```
# Create project directory first
mkdir mychatbot && cd mychatbot && git init

# Launch OpenCode
opencode
```

Inside TUI:

```
/connect
# Select: groq
# Paste: your GROQ_API_KEY

/models
# Select: groq/llama-3.3-70b-versatile
```

Pro Tip: You can also use free (NO API KEY) model `opencode/big-pickle` or any other model you wish. OpenCode lets you change models mid-session with `/models`.

Step 2 — Verify the connection:

```
# In TUI:
Say "Hello, which model are you?"
```

Stage 1: Project Bootstrap[0.5em]

Teaching OpenCode about your project

Run /init — Generate AGENTS.md

The first command in any new project. Scans the directory and writes the AI's instruction manual.

```
# Inside TUI (project is still empty, that's fine):
/init
```

OpenCode scans the directory. Since it's empty, it will ask you to describe the project. Tell it:

```
This is a Python project called mychatbot.
It provides:
  1. A CLI chatbot (cli.py) using LangChain + Groq
     LLM
  2. A Streamlit web chatbot (app.py)
  3. A reusable chain module (chain.py)
Tech stack: Python 3.11, LangChain, langchain-groq,
  Streamlit, Rich, pytest.
Coding conventions: black formatter, type hints,
  docstrings on all public functions.
```

OpenCode writes `AGENTS.md` — commit it to Git. Every future session reads it automatically.

```
git add AGENTS.md && git commit -m "chore: add
    AGENTS.md"
```

Inspect and Edit AGENTS.md

Open `AGENTS.md` and extend it with project-specific rules:

```
## Project: mychatbot

## Tech Stack
- Python 3.11
- LangChain 0.3+, langchain-groq
- Streamlit 1.35+
- Rich (CLI output)
- pytest + pytest-mock

## Build Commands
- Install: `pip install -e ".[dev]"`
- Tests: `pytest tests/ -v`
- Format: `black mychatbot/ tests/`
- Lint: `ruff check mychatbot/`

## Conventions
- Type hints on all functions
- Docstrings (Google style) on all public functions
- No hardcoded API keys:  always read from env vars
- Tests in tests/ mirroring mychatbot/ structure
```

```
- Keep chain.py model-agnostic (injected via
    config)

## File Map
- mychatbot/chain.py  :  LangChain chain factory
- mychatbot/cli.py    :  CLI entry point (Rich)
- mychatbot/app.py    :  Streamlit chatbot UI
- tests/            :  pytest test suite
```

Stage 2: Requirements & Specification[0.5em] Using OpenCode to write specs before code

Setup: Specs Custom Agent

Create a dedicated **Specs agent**: read-only, focused on producing Markdown docs:

```
# Create the agents directory
mkdir -p .opencode/agents

# Create the Specs agent file
cat > .opencode/agents/specs.md << 'EOF'
---
name: specs
description: >
  Technical specification writer. Produces clear,
  structured Markdown spec docs from user stories
  or feature descriptions. Read-only:  never
    modifies
  source code. Model: claude-sonnet for quality
    writing.
model: anthropic/claude-sonnet-4-5
mode: subagent
tools:
  read: true
  write: true       # can write .md files to docs/
  bash: false
  edit: false
temperature: 0.3
---
You are a senior technical writer and software
    architect.
When asked to write a spec, produce a Markdown
    document
covering: Overview, Goals, Non-Goals, User Stories,
Functional Requirements, Non-Functional
    Requirements,
Data Models, API/Interface Contracts, and Open
    Questions.
Always save output to docs/specs/<feature>.md.
EOF
```

Generate the Project Spec with @specs

Switch to **Plan mode** first (Tab key), then invoke the Specs subagent:

```
# In TUI (Plan mode, Tab to switch):
@specs Write a technical specification for
    mychatbot.
It should cover:
- The CLI chatbot interface (multi-turn
    conversation,
  history, exit command)
- The Streamlit web chatbot (session state,
    streaming)
- The shared LangChain chain module (model config,
  system prompt injection, memory management)
- Environment variable requirements
- Error handling expectations
Save to docs/specs/mychatbot-spec.md
```

Result: `docs/specs/mychatbot-spec.md`: a structured specification that will guide implementation.

Key Point: Writing the spec first catches design issues before any code is written. The spec also becomes part of your project context for future OpenCode sessions.

```
git add docs/ && git commit -m "docs: add
    mychatbot spec"
```

Expected Spec Output (docs/specs/mychatbot-spec.md)

The Specs agent should produce a document like:

```
# mychatbot Technical Specification

## Overview
mychatbot is a Python package providing two
    interfaces
for multi-turn LLM conversations via Groq's API.

## User Stories
- As a developer, I want a CLI chatbot so I can
    test
  prompts without leaving the terminal.
- As a user, I want a Streamlit UI for a
    browser-based
  conversation experience.

## Functional Requirements
### chain.py
- FR-1: Accept model name and system prompt as
    config
- FR-2: Maintain conversation history
    (ConversationBuffer)
- FR-3: Return streaming responses

### cli.py
- FR-4: Read GROQ_API_KEY from environment
- FR-5: Display user/assistant turns with Rich
    formatting
- FR-6: Exit on /exit or Ctrl+C

## Data Models
### Message
- role: Literal["user", "assistant", "system"]
- content: str

## Open Questions
- OQ-1: Should we support multi-model routing?
```

Stage 3: Architecture Design[0.5em] Plan mode + workspace context

Use Plan Mode to Design Architecture

Stay in **Plan mode** (Tab). Use `@workspace` to give OpenCode full project context:

```
# In TUI (Plan mode):
@workspace Based on docs/specs/mychatbot-spec.md,
propose the directory structure and module design
for mychatbot. Specifically:
1. What goes in chain.py? Show the class interface.
2. How should cli.py call chain.py?
3. How should app.py manage Streamlit session
    state?
4. What should pyproject.toml look like?
Do NOT write any code yet:  just describe the plan.
```

What OpenCode does: Reads the spec, proposes a coherent architecture. You review and iterate on the plan before any files are created.

Typical plan output includes:

- Class interfaces with method signatures
- Dependency injection patterns (no hardcoded model in chain)
- Streamlit `st.session_state` design
- pyproject.toml entry point declaration

If you disagree with any part, say so and iterate!

```
Actually, use a functional API in chain.py (not a
    class).
Return a callable that takes messages and returns
    a string.
```

Setup: Project Rules File (Global Coding Standards)

Before writing code, set up a **global rules file**: personal standards that apply to all your projects:

```
mkdir -p ~/.config/opencode
cat > ~/.config/opencode/AGENTS.md << 'EOF'
## My Personal Coding Standards (Global Rules)

### Python
- Always use type hints. No untyped functions.
- Use f-strings, not .format() or %
- Prefer pathlib.Path over os.path
- Never catch bare `except:`:  always specify
    exception type
- Every module must have a module-level docstring

### Git
- Commit messages: conventional commits format
  (feat:, fix:, chore:, docs:, test:, refactor:)
- No direct commits to main:  always branch

### Testing
- One test file per source file (test_chain.py for
    chain.py)
- Use pytest fixtures, not setUp/tearDown
- Cover happy path, edge cases, and error cases

### Output
- When generating code, show the full file content
- When editing, always confirm what changed and why
EOF
```

These rules apply to *every* project you work on with OpenCode.

Stage 4: Implementation[0.5em] Build mode: writing the actual code

Create Project Scaffold

Switch to **Build mode** (Tab). Scaffold the project structure:

```
# In TUI (Build mode):
Create the project scaffold for mychatbot:
- pyproject.toml with: name=mychatbot,
    python>=3.10,
  dependencies=[langchain, langchain-groq,
    streamlit,
  rich], dev deps=[pytest, pytest-mock, black,
    ruff, faker]
  and entry point: mychatbot=mychatbot.cli:main
- mychatbot/__init__.py  (version = "0.1.0")
- mychatbot/chain.py     (empty module with
    docstring)
- mychatbot/cli.py       (empty module with
    docstring)
- mychatbot/app.py       (empty module with
    docstring)
- tests/__init__.py
- tests/test_chain.py  (empty test file)
- tests/test_cli.py    (empty test file)
- .gitignore           (Python standard)
```

After scaffold:

```
pip install -e ".[dev]"
git add . && git commit -m "chore: project
    scaffold"
```

Implement chain.py

```
# In TUI (Build mode):
@file:mychatbot/chain.py
Implement the LangChain chain module. Requirements:
- Function: make_chain(model_name: str,
    system_prompt: str = "You are a helpful
    assistant",
    temperature: float = 0.7) -> callable
- The returned callable takes list[dict] (messages
    with
  role/content keys) and returns str
- Use ChatGroq from langchain_groq
- Use ChatPromptTemplate with system + human
    messages
- Support conversation history: prepend all prior
    messages
- Read GROQ_API_KEY from os.environ (raise clear
    error
  if missing)
- Include full type hints and Google-style
    docstring
- Handle groq API errors with informative messages
```

Review the generated code:

```
# Ask OpenCode to explain its choices:
Why did you use ChatPromptTemplate instead of
MessagesPlaceholder? Is there a better approach
for multi-turn history?
```

Commit after review:

```
git add mychatbot/chain.py
git commit -m "feat: implement LangChain chain
    factory"
```

Implement cli.py

```
# In TUI (Build mode):
@file:mychatbot/cli.py
Implement the CLI chatbot. Requirements:
- main() function (entry point)
- Use Rich: Console for output, Panel for welcome
    message,
  Markdown for rendering assistant responses
- Show user input with a styled [You] prompt
- Show assistant output with [mychatbot] prefix in
    green
- Maintain conversation history across turns as
  list[dict] with role/content
- Commands: /exit or /quit to end, /clear to reset
    history
- Ctrl+C gracefully exits with "Goodbye!" message
- Call make_chain() from mychatbot.chain
- Model and system prompt configurable via env
    vars:
  mychatbot_MODEL (default:
    llama-3.3-70b-versatile)
  mychatbot_SYSTEM_PROMPT (default: helpful
    assistant)
- Show token count or turn count if available
```

```
git add mychatbot/cli.py
git commit -m "feat: implement Rich CLI chatbot"
```

Implement app.py (Streamlit)

```
# In TUI (Build mode):
@file:mychatbot/app.py
Implement the Streamlit chatbot app. Requirements:
- st.set_page_config: title="mychatbot",
    layout="centered"
- st.session_state["messages"] for conversation
    history
- st.session_state["chain"] to cache the chain
    callable
  (init once, reuse across reruns)
- Display chat history using st.chat_message
    bubbles
- st.chat_input for user input
- Show a spinner while waiting for the model
    response
- Sidebar: model selector dropdown
  (llama-3.3-70b-versatile, llama-3.1-8b-instant,
   mixtral-8x7b-32768) and system prompt text area
- "Clear conversation" button in sidebar
- Handle missing GROQ_API_KEY with st.error and
    st.stop()
```

```
# Quick test:
streamlit run mychatbot/app.py

git add mychatbot/app.py
git commit -m "feat: implement Streamlit chatbot
    UI"
```

Demo Checkpoint: Run the Chatbot

Verify both interfaces work before moving to testing:
CLI:

```
mychatbot
# or: python -m mychatbot.cli

# Try:
You: What is LangChain?
You: Can you give me a code example?
You: /clear
You: /exit
```

Streamlit:

```
streamlit run mychatbot/app.py
# Open: http://localhost:8501
# Change model in sidebar
# Type a question, observe streaming
# Click "Clear conversation"
```

If something breaks: Use `/undo` in OpenCode to revert the last change, rephrase your instruction, and retry. OpenCode will show you exactly what changed.

Stage 5: Testing with Synthetic Data[0.5em] Custom commands + skills for test generation

Create a Testing Skill

Skills are loaded on-demand. Create a `pytest-patterns` skill for consistent test generation:

```
mkdir -p .opencode/skills/pytest-patterns
cat > .opencode/skills/pytest-patterns/SKILL.md <<
    'EOF'
---
name: pytest-patterns
description: >
  Best practices and patterns for writing pytest
    tests
  for this project. Use when generating or
    reviewing
  Python unit tests.
license: MIT
---
## Pytest Patterns for mychatbot

### Fixtures
- Use `@pytest.fixture` for shared setup
- `mock_chain` fixture: returns a MagicMock
    callable
  that returns "Mocked response"
- `sample_messages` fixture: list of
    user/assistant dicts

### Mocking
- Mock `mychatbot.chain.ChatGroq` to avoid real
    API calls
- Use `pytest.raises` for exception testing
- Use `monkeypatch` for env var testing

### Synthetic Data (Faker)
- Import `faker.Faker` for realistic fake data
- Generate fake user messages, conversation
    histories
- Use `@pytest.mark.parametrize` for data-driven
    tests

### Coverage Target: 85%+
EOF
```

Create the /test-gen Custom Command

Custom commands = reusable prompt shortcuts. Create one for test generation:

```
mkdir -p .opencode/commands
cat > .opencode/commands/test-gen.md << 'EOF'
---
description: >
  Generate comprehensive pytest tests for a source
    file.
  Uses pytest-patterns skill. Pass the source file
    path
  as the argument.
agent: build
---
Generate comprehensive pytest tests for $ARGUMENTS.

Before writing tests:
1. Read the pytest-patterns skill for this project
2. Read the source file to understand all
    functions/classes
3. Read the existing test file (if any) to avoid
    duplicates

Write tests covering:
- Happy path for each public function
- Edge cases (empty input, None, boundary values)
- Error cases (missing env vars, invalid model
    names,
  API failures:  mock the LLM calls, no real API
    use)
- Use Faker to generate synthetic conversation
    data for
  multi-turn history tests

Output: complete updated test file, no
    placeholders.
EOF
```

Usage:

```
/test-gen mychatbot/chain.py
/test-gen mychatbot/cli.py
```

Generate Tests with Synthetic Data

Run the custom command:

```
# In TUI (Build mode):
/test-gen mychatbot/chain.py
```

OpenCode will generate something like:

```
# tests/test_chain.py
import pytest
from unittest.mock import patch, MagicMock
from faker import Faker
from mychatbot.chain import make_chain

fake = Faker()

@pytest.fixture
def mock_chat_groq():
    with patch("mychatbot.chain.ChatGroq") as mock:
        instance = mock.return_value
        instance.invoke.return_value.content =
    "Hello!"
        yield mock

@pytest.fixture
def sample_conversation():
    """Synthetic multi-turn conversation using
    Faker."""
    return [
        {"role": "user",    "content":
    fake.sentence()},
        {"role": "assistant","content":
    fake.paragraph()},
        {"role": "user",    "content":
    fake.question()},
```

```
    ]

def
     test_make_chain_returns_callable(mock_chat_groq,
     monkeypatch):
    monkeypatch.setenv("GROQ_API_KEY", "test-key")
    chain = make_chain("llama-3.3-70b-versatile")
    assert callable(chain)
```

Generate Tests for cli.py and Run Suite

```
# In TUI:
/test-gen mychatbot/cli.py
```

CLI tests will cover:

- `/exit` command terminates the loop
- `/clear` resets the conversation history
- Missing `GROQ_API_KEY` raises `EnvironmentError`
- `KeyboardInterrupt` (Ctrl+C) exits gracefully
- Rich console output contains expected text

Run the full test suite:

```
pytest tests/ -v --tb=short
# Expected: all green, no real API calls made

# With coverage:
pytest tests/ --cov=mychatbot
     --cov-report=term-missing
# Target: >85% coverage
```

If tests fail:

```
# In TUI:
The test test_make_chain_missing_key is failing
     with:
KeyError: 'GROQ_API_KEY'. Paste the traceback.
OpenCode reads the test + source and proposes the
     fix.
```

```
git add tests/ && git commit -m "test: add unit
     tests with synthetic data"
```

Create a /test-run Command (Bonus)

Automate full test + coverage + report in one command:

```
cat > .opencode/commands/test-run.md << 'EOF'
---
description: Run the full test suite and summarize
     results
agent: build
---
Run the full test suite for this project:

1. Execute: `pytest tests/ -v --tb=short
     --cov=mychatbot
   --cov-report=term-missing 2>&1`
2. Parse the output and give me:
   - Pass/fail count
   - Coverage percentage per module
   - List of any failing tests with the error
     message
   - Three specific suggestions to improve coverage
     for any module below 85%
EOF
```

Usage:

```
/test-run
```

This is the power of custom commands: complex multi-step workflows reduced to a single slash command. The entire team can use the same command once it's committed to Git.

Stage 6: Debugging[0.5em] Debug agent + /undo workflow

Create a Debug Agent

A Debug agent has **read + bash only**: it investigates, never modifies:

```
cat > .opencode/agents/debug.md << 'EOF'
---
name: debug
description: >
  Investigates bugs and runtime errors. Read-only:
  never modifies files. Uses bash to run tests and
  inspect state. Outputs a structured findings
     report.
model: anthropic/claude-sonnet-4-5
mode: subagent
tools:
  bash: true
  read: true
  grep: true
  glob: true
  write: false
  edit: false
temperature: 0.1
steps: 20
---
You are a meticulous debugger. When given an error:
1. Read relevant source files
2. Trace the error from the stack trace
3. Run targeted bash commands to reproduce the
     issue
4. Identify the root cause
5. Suggest a minimal fix (do NOT apply it:  report
     only)

Output format:
## Root Cause
## Affected Files
## Minimal Reproduction
## Suggested Fix
EOF
```

Debug Workflow Demo

Scenario: The CLI crashes when the user types a very long message.

Step 1: Invoke debug agent:

```
# In TUI:
@debug The CLI chatbot crashes when the user
     enters a
message longer than ~4000 characters. Here's the
     error:

groq.BadRequestError: Error code: 400
  Request too large for model
  llama-3.3-70b-versatile in 5 requests per minute

Stack trace:
  File "mychatbot/chain.py", line 34, in __call__
    response = self.llm.invoke(messages)
```

Debug agent output: Root cause identified: no token limit validation before sending to API.

Step 2: Apply fix in Build mode:

```
# Switch to Build mode (Tab), then:
@file:mychatbot/chain.py
Add input validation: if the total character count
     of
all messages exceeds 12000 chars, truncate the
     oldest
messages (not the system message) until it fits.
Raise a warning via the logging module when
     truncation occurs.
```

Step 3: If the fix is wrong, use /undo:

```
/undo  # reverts the edit, restores previous state
# Rephrase and retry
```

Stage 7: Refactoring[0.5em] Using rules and skills for consistent improvements

Create a Refactoring Skill

```
mkdir -p .opencode/skills/refactor-patterns
cat > .opencode/skills/refactor-patterns/SKILL.md
    << 'EOF'
---
name: refactor-patterns
description: >
  Patterns and criteria for safe refactoring of
    Python code.
  Use when asked to refactor or improve code
    quality.
---
## Refactoring Rules

### Safety First
- Never change public API signatures without
    updating
  all callers and the spec doc
- Run tests before and after to confirm no
    regression

### Python Improvements to Look For
- Replace dict literals for message objects with
  a TypedDict or dataclass
- Extract magic strings (model names, env var
    names)
  into constants at module level
- Replace print() with logging module
    (configurable level)
- Move configuration into a Config dataclass
  (not scattered kwargs)

### Code Smell Checklist
- Functions longer than 40 lines: consider
    splitting
- Duplicate code across cli.py and app.py: extract
    to utils
- Nested if/else depth > 3: refactor to early
    returns
EOF
```

Refactor: Extract Shared Config

```
# In TUI (Build mode):
@workspace
Read the refactor-patterns skill, then:
1. Identify duplicated configuration logic between
   cli.py and app.py (both read mychatbot_MODEL and
   mychatbot_SYSTEM_PROMPT from env vars)
2. Create mychatbot/config.py with a Config
    dataclass
   that reads all env vars in one place
3. Update cli.py and app.py to import and use
    Config
4. Add a test for Config in tests/test_config.py
   covering defaults and env var overrides

Use @file: to read each file before editing.
Show me the plan before making any changes.
```

Review the plan, then approve to proceed:

```
# If plan looks good:
Looks good. Go ahead and implement it.

# After implementation, run tests:
pytest tests/ -v
```

```
git add . && git commit -m "refactor: extract
    Config dataclass"
```

Refactor: Add Structured Logging

```
# In TUI (Build mode):
@workspace
Replace all print() debug statements in mychatbot/
with Python's logging module:
- Create a logger = logging.getLogger(__name__)
  at the top of each module
- Use logging.INFO for user-facing info
- Use logging.DEBUG for internal state
- Use logging.ERROR for caught exceptions
- In cli.py, configure basicConfig with level from
  env var LOG_LEVEL (default: WARNING)
- Do NOT change any Rich console output:  that
    stays
Only replace bare print() debug statements.
```

Point to note: The global rules file (AGENTS.md) already says "Never catch bare except": OpenCode will respect this without being told again.

```
# Verify nothing broke:
pytest tests/ -v
mychatbot  # quick manual smoke test

git add . && git commit -m "refactor: replace
    print with logging"
```

Stage 8: Documentation[0.5em] Docs agent + auto-generated README

Create a Docs Agent

```
cat > .opencode/agents/docs.md << 'EOF'
---
name: docs
description: >
  Documentation writer. Generates README, API docs,
  usage guides, and changelog entries. Can write
  Markdown files but cannot modify Python source.
model: anthropic/claude-sonnet-4-5
mode: subagent
tools:
  read: true
  write: true
  bash: false
  edit: false
temperature: 0.4
---
You are a technical documentation expert. When
    asked
to generate docs:
- Read all relevant source files first
- Use the existing spec in docs/specs/ as the
    source
  of truth for intended behavior
- Write clear, copy-pasteable code examples
- Include a "Quick Start" section in every README
- Use GitHub-flavored Markdown
- Never invent features not present in the source
    code
EOF
```

Generate README and API Docs

```
# In TUI:
@docs Generate a complete README.md for mychatbot:
- Badges: Python version, license (MIT), tests
    passing
- Description: one paragraph
- Features: bullet list
- Installation: pip install, env var setup
- Quick Start: CLI example and Streamlit launch
    command
- Configuration: table of all env vars with
    defaults
- Development: how to run tests, format, lint
- Contributing section
- License section
```

```
Then generate docs/api.md covering the public API
of chain.py and config.py with usage examples.
```

```
# Also create a /docs command for future use:
cat > .opencode/commands/docs.md << 'EOF'
---
description: Generate/update documentation for a
     module
agent: docs
---
Generate complete documentation for $ARGUMENTS.
Read the source file, update or create the
     corresponding
doc in docs/. Follow the existing README style.
EOF
```

```
git add README.md docs/ && git commit -m "docs:
     generate README and API docs"
```

Generate CHANGELOG and Version Bump

```
# In TUI:
@docs Generate a CHANGELOG.md for mychatbot in
Keep-a-Changelog format.
Run `git log --oneline` to get the commit history,
then group commits into sections:
- Added (feat: commits)
- Fixed (fix: commits)
- Changed (refactor: commits)
- Documentation (docs: commits)
Tag this as version 0.1.0 with today's date.
```

Creating a custom /changelog command:

```
cat > .opencode/commands/changelog.md << 'EOF'
---
description: Update CHANGELOG from recent git
     commits
agent: docs
---
Run `git log --oneline --since="$ARGUMENTS"` to get
recent commits. Update CHANGELOG.md with a new
version section grouping commits by type:
feat->Added, fix->Fixed, refactor->Changed,
     docs->Docs.
Suggest a semantic version bump based on the
     changes.
EOF

# Usage: /changelog "1 week ago"
# Usage: /changelog "last tag"
```

Stage 9: Code Review[0.5em] Review agent + permissions

Create a Review Agent with Strict Permissions

```
cat > .opencode/agents/review.md << 'EOF'
---
name: review
description: >
  Code reviewer. Performs a thorough code review
     covering
  correctness, security, performance, style, and
     test
  coverage. Read-only:  cannot modify any files.
model: anthropic/claude-sonnet-4-5
mode: subagent
tools:
  read: true
  grep: true
  glob: true
  bash: false
  write: false
  edit: false
permission:
  "*": deny  # deny everything not explicitly
     listed
  "read": allow
  "grep": allow
  "glob": allow
temperature: 0.2
steps: 30
---
Perform a code review. For each file, check:
1. Correctness: logic bugs, off-by-one, type errors
2. Security: hardcoded secrets, injection,
     unvalidated input
3. Performance: unnecessary loops, blocking calls
4. Style: matches project conventions in AGENTS.md
5. Test coverage: untested branches or edge cases
Output: structured report with severity
     (Critical/Major/Minor)
EOF
```

Run a Full Code Review

```
# In TUI:
@review Review the entire mychatbot codebase.
Focus on:
1. Security: are API keys ever logged or printed?
2. Error handling: are all Groq API errors caught?
3. Streamlit: any session state race conditions?
4. Tests: what branches are not covered?
5. Type correctness: any missing type hints?

For each issue, note:
- File and line number
- Severity: Critical / Major / Minor
- What the issue is
- How to fix it
```

Apply review findings:

```
# After reading the review report:
Apply all Critical and Major fixes identified in
the review report. For each fix, show what changed.
```

```
# Create a /review command:
cat > .opencode/commands/review.md << 'EOF'
---
description: Full code review of $ARGUMENTS (or
     workspace)
agent: review
---
Review $ARGUMENTS for correctness, security,
performance, and style. Group by severity.
EOF
```

Stage 10: GitHub Workflows via MCP[0.5em] Connecting OpenCode to external services

Configure the GitHub MCP Server

MCP (Model Context Protocol) lets OpenCode call external tools. GitHub MCP gives it access to repos, issues, and PRs.

Step 1: Create project opencode.json with MCP config:

```
cat > .opencode/opencode.json << 'EOF'
{
  "mcp": {
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y",
     "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_TOKEN": "$GITHUB_TOKEN"
```

```
      }
    }
  },
  "permission": {
    "github_*": "ask",
    "bash": {
      "git push *": "ask",
      "git commit *": "allow",
      "git add *": "allow"
    }
  }
}
EOF
```

Step 2: Create a GitHub repo and push:

```
# Create repo on GitHub (manually or via gh CLI)
gh repo create mychatbot --public --source=. --push
```

GitHub MCP Workflows

With the GitHub MCP server active, OpenCode can manage your repo:

Create issues from code review findings:

```
# In TUI:
Create GitHub issues for each Critical and Major
     finding
from the code review we just ran. Title each issue
clearly, add labels: "bug" for Critical,
     "enhancement"
for Major. Assign them all to me.
```

Create a feature branch and PR:

```
Create a feature branch called "feat/add-streaming"
then implement streaming responses in chain.py
     using
LangChain's streaming callback. When done, create a
GitHub PR with:
- Title: "feat: add streaming responses"
- Body: description of changes, testing done,
  link to relevant issue
```

Auto-respond to PR comments:

```
# Non-interactive mode (usable in CI/GitHub
     Action):
opencode run "Read the latest unresolved PR comment
on PR #1 and either apply the requested change or
post a response explaining why we shouldn't."
```

Configure Additional MCP Servers

OpenCode supports many MCP servers. Add them to `opencode.json`:

```
{
  "mcp": {
    "github": { /* ... as before ... */ },

    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y",
     "@modelcontextprotocol/server-filesystem",
                "/home/user/mychatbot"]
    },

    "brave-search": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y",
     "@modelcontextprotocol/server-brave-search"],
      "env": { "BRAVE_API_KEY": "$BRAVE_API_KEY" }
    }
  }
}
```

With Brave Search MCP, you can:

```
# In TUI:
Search for the latest LangChain 0.3 changelog and
tell me if any of our chain.py code needs updating.
```

Bonus: Advanced Techniques[0.5em]

Plugins, variants, and CI integration

Model Variants: Use the Right Model for the Task

Define model variants so you can pick the right model for each stage:

```
# ~/.config/opencode/opencode.json (global config)
{
  "model": "anthropic/claude-sonnet-4-5",
  "modelVariants": {
    "fast": {
      "model": "groq/llama-3.1-8b-instant",
      "temperature": 0.1
    },
    "smart": {
      "model": "anthropic/claude-sonnet-4-5",
      "temperature": 0.3
    },
    "local": {
      "model": "ollama/llama3.2",
      "temperature": 0.5
    }
  }
}
```

Switch variants: `/models` → pick from list, or `opencode -m groq/llama-3.1-8b-instant`

Practical guide:

- **fast/local**: test generation, simple edits, quick questions
- **smart**: architecture design, complex refactoring, security review
- **local (Ollama)**: sensitive codebases, air-gapped environments

Non-Interactive Mode in CI/CD

OpenCode's `opencode run` enables AI-powered CI workflows:

GitHub Actions example (.github/workflows/ai-review.yml):

```
name: AI Code Review
on: [pull_request]

jobs:
  review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm install -g opencode-ai
      - name: Run AI review on changed files
        env:
          ANTHROPIC_API_KEY: ${{
     secrets.ANTHROPIC_API_KEY }}
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
        run: |
          CHANGED=$(git diff --name-only
     origin/main HEAD \
                    | grep '\.py$' | tr '\n' ' ')
          opencode run "Review these changed
     Python files
            for bugs and style issues: $CHANGED.
            Post findings as a GitHub PR comment
            using the github MCP tool."
```

Other CI uses:

- Auto-generate test stubs for new functions in a PR
- Update CHANGELOG on every merge to main
- Check that new public functions have docstrings

Custom Tool: Python Linting Integration

Create a custom TypeScript tool that wraps Python tools:

```
// .opencode/tools/python-lint.ts
import { tool } from "@opencode-ai/plugin"
import { execSync } from "child_process"

export const lint = tool({
  description: "Run ruff linter on a Python file
    or directory",
  args: {
    path: tool.schema.string().describe(
      "File or directory to lint (relative to
     project root)"
    )
  },
  async execute({ path }, context) {
    try {
      const result = execSync(
        `ruff check ${path} --output-format=json`,
        { cwd: context.worktree, encoding: "utf8" }
      )
      const issues = JSON.parse(result)
      return issues.length === 0
        ? "No lint issues found."
        : `Found ${issues.length} issues:\n` +
          issues.map((i: any) =>
            `  ${i.filename}:${i.location.row}
     [${i.code}] ${i.message}`
          ).join("\n")
    } catch (e: any) {
      return e.stdout || e.message
    }
  }
})
```

Usage: `Lint mychatbot/ and fix all E501 (line too long) errors`

Session Sharing and Team Collaboration

OpenCode sessions can be shared as read-only URLs:

```
# Share your debugging session:
/share
# Returns: https://opencode.ai/s/abc123...

# Team member opens it in their browser and sees
    the
# full conversation: the bug, the debug agent's
    analysis,
# the fix applied, and the tests that now pass.
```

Practical team uses:

- Share a debugging session in Slack instead of a long explanation
- Attach a session URL to a GitHub issue as context
- Onboarding: share a "How I navigate this codebase" session
- Architecture decision records: the conversation *is* the ADR

Resuming shared patterns:

```
# Keep a known-good session for common workflows
opencode run --title "mychatbot-debug-template" \
  "Read AGENTS.md and docs/specs/mychatbot-spec.md.
   You are now ready to debug mychatbot issues."

# Resume it anytime:
opencode -c  # resumes most recent
opencode session list | grep mychatbot-debug
opencode session continue <id>
```

Workshop Summary

What We Built

Project `mychatbot/`: complete from scratch:

- `mychatbot/chain.py`: LangChain + Groq chain factory
- `mychatbot/cli.py`: Rich CLI chatbot with history
- `mychatbot/app.py`: Streamlit multi-turn chatbot
- `mychatbot/config.py`: Centralized env-based config
- `tests/`: Full pytest suite with synthetic Faker data
- `docs/specs/mychatbot-spec.md`: Technical specification
- `README.md` + `docs/api.md`: Auto-generated docs
- `CHANGELOG.md`: Conventional commit-based changelog

OpenCode config committed to repo:

- `AGENTS.md`: Project intelligence file
- `.opencode/agents/`: specs, debug, docs, review agents
- `.opencode/skills/`: pytest-patterns, refactor-patterns
- `.opencode/commands/`: /test-gen, /test-run, /review, /docs, /changelog
- `.opencode/tools/python-lint.ts`: Custom lint tool
- `.opencode/opencode.json`: MCP GitHub + permissions

OpenCode SDLC Cheat Sheet

Task	OpenCode Command/Technique
Bootstrap project	`/init` → AGENTS.md
Write spec	`@specs` subagent
Design architecture	Plan mode + `@workspace`
Write code	Build mode + `@file:`
Generate tests	`/test-gen <file>` (custom cmd)
Run test suite	`/test-run` (custom cmd)
Debug an error	`@debug` subagent
Undo a bad change	`/undo`
Refactor safely	refactor-patterns skill
Generate README	`@docs` subagent
Update CHANGELOG	`/changelog "last week"`
Code review	`@review` subagent
Create GitHub issue	GitHub MCP
Create PR	GitHub MCP
Lint code	python-lint custom tool
Share a session	`/share`
Resume a session	`opencode -c`

Key Lessons from the Workshop

- **AGENTS.md is your multiplier.** Write it carefully and every session starts smarter. Commit it to Git: your whole team benefits.
- **Plan before Build.** For anything non-trivial, use Plan mode to review the approach. Avoid costly rewrites.
- **Use `@mentions` to be precise.** `@file:` focuses the agent; `@workspace` gives it breadth. Use the right scope.
- **Custom commands save time.** Once you've typed the same prompt three times, make it a `/command`. Share it via Git.
- **Skills keep context lean.** Don't put everything in AGENTS.md. Lazy-load test patterns, refactor rules, and domain knowledge via skills.
- **Agents enforce boundaries.** A read-only Review agent cannot accidentally delete code. Permissions are your safety net.
- **MCP extends reach.** GitHub, databases, search: connect them once and the agent can act across your entire stack.
- **`/undo` is your safety net.** Never fear a bad edit. One command reverts it.

Exercises for Self-Study

- **Exercise 1:** Add a `/exit` history command to cli.py that saves conversation to `/.mychatbot_history.json`. Use `/test-gen` to cover it.
- **Exercise 2:** Create a "Security" agent that only checks for API key leaks, SQL injection patterns, and unvalidated inputs. Test it on an intentionally buggy snippet.
- **Exercise 3:** Add a PostgreSQL MCP server and a `/db-query` custom command. Have the chatbot store conversation logs in a database.
- **Exercise 4:** Implement a multi-model skill that routes short queries to `llama-3.1-8b-instant` and complex queries to `claude-sonnet-4-5`.
- **Exercise 5:** Set up the GitHub Action for AI code review on your own repository. Trigger a PR and observe the auto-review.
- **Exercise 6:** Use `opencode run` in non-interactive mode to auto-generate release notes from the last 10 commits and post them as a GitHub release.

Resources

- **OpenCode Docs:** `opencode.ai/docs`
- **GitHub:** `github.com/opencode-ai/opencode`
- **LangChain + Groq:** `python.langchain.com/docs/integrat`
- **Groq Console (API keys):** `console.groq.com`
- **MCP Server Registry:** `github.com/modelcontextprotocol`
- **Streamlit Docs:** `docs.streamlit.io`
- JP Caparas, "The definitive guide to OpenCode", Dev Genius / reading.sh, Feb 2026.
- NXCode.io, "OpenCode Tutorial 2026: Install & Setup After Anthropic Block."

Workshop complete. You now have a production-ready OpenCode workflow.[0.5em] **Happy coding!**

Copyleft ㋯ Send suggestions to yogeshkulkarni@yahoo.com